

ShopEase — Docker & PostgreSQL Runbook

Containerising an Angular + ASP.NET Core app, then migrating SQL Server → PostgreSQL · 24 August 2026

This document records every change made to the ShopEase project, in the order it was made, and gives you the commands to reproduce it. Every file listed was written to disk; every command shown was run; every verification result quoted is real output, not an expectation.

Two pieces of work, in sequence:

- Part I — Containerisation.** Three services under Docker Compose. Two source edits, seven new files.
- Part II — PostgreSQL migration.** Driven by a 125-finding audit. Provider swap, regenerated schema, and the case-sensitivity work that nothing would have told you about.

What you are building

Service	Image	Role
db	postgres:17-alpine	PostgreSQL. Pulled ready-made, never built.
api	shopease-api:1.0	ASP.NET Core 10. Built from Backend/Dockerfile .
web	shopease-web:1.0	nginx serving the compiled Angular bundle.

The key idea. Only **one** port matters to a user: 4200 . The browser asks nginx for everything. When it wants data it calls a relative path like /api/products ; nginx forwards that to the api container internally. The browser never learns the API's address — and because every request comes from one origin, **CORS disappears entirely**.

Before you start

Disk space	~8 GB free. .NET SDK 3.0 GB (build only) + build cache ~1.5 GB + Postgres 0.25 GB + the rest. Windows itself wants 15–20 GB free to stay healthy.
Docker Desktop	Installed and running . Verify with docker version .
Ports free	4200 , 5110 , 5432
Project root	C:\Users\1404\Downloads\shopease

Learn from our failure. The first build attempt died because the drive hit **0 bytes free of 238 GB**. Docker's storage flipped read-only mid-extraction, the engine began returning 500 Internal Server Error , and it eventually shut down entirely. Check free space *before* building, not after.

Part I — Containerisation

Two edits to your source code

1 Point the frontend at a relative API path

File: `src/ShopEase.Web/src/environments/environment.ts` · **Backup:** `environment.ts.bak`

```
- apiUrl: 'http://localhost:5110/api',  
+ apiUrl: '/api',
```

Why. An absolute URL hard-codes where the API lives. Inside Docker that address is wrong, and it forces the browser into a cross-origin request needing CORS. A relative `/api` means "same host that served this page" — which is nginx, which knows how to reach the API.

Safe for your auth interceptor. It gates on `req.url.startsWith(environment.apiUrl)`. A request built as `/api/products` still starts with `/api`, so JWT attachment keeps working unchanged.

2 Run migrations at startup, drop HTTPS redirect

File: `Backend/ShopEase.Api/Program.cs` · **Backup:** `Program.cs.bak`

```
line 176 using (var scope = app.Services.CreateScope())  
line 179     await db.Database.MigrateAsync();  
  
line 196 // app.UseHttpsRedirection();
```

Why migrations move. The container starts with an empty database. Running `MigrateAsync()` at boot means the API builds its own schema on first start — no manual step. **Why the redirect goes.** Kestrel serves plain HTTP inside the container; TLS is nginx's job at the edge. Leaving it on makes the API bounce internal requests to a port that is not listening.

Seven files created

File	Purpose
<code>docker-compose.yml</code>	Root. All three services, the network, the volume.
<code>.dockerignore</code>	Root.
<code>Backend/Dockerfile</code>	Two-stage .NET build: SDK compiles, runtime ships.
<code>Backend/.dockerignore</code>	Excludes <code>bin</code> , <code>obj</code> from the API build context.
<code>src/ShopEase.Web/Dockerfile</code>	Two-stage Angular build: Node compiles, nginx ships.
<code>src/ShopEase.Web/.dockerignore</code>	Excludes <code>node_modules</code> , <code>dist</code> .
<code>src/ShopEase.Web/nginx.conf</code>	SPA fallback + the <code>/api</code> reverse proxy.

Why two-stage builds matter

Build stage (thrown away)	Runtime stage (shipped)
.NET SDK — ~3.0 GB compiler, NuGet cache	ASP.NET runtime — ~230 MB
Node + npm — ~1.1 GB node_modules , Angular CLI	nginx + static files — ~50 MB

Only the final stage becomes the image. The shipped web image contains **no Node, no npm, no node_modules** — just compiled files and a web server.

The .dockerignore rule that cost us a build

.dockerignore applies only to its own build context directory. Docker looks for the file *inside the context* — not at the repository root, and not next to `docker-compose.yml`. Our contexts are `./Backend` and `./src/ShopEase.Web`, so a single root `.dockerignore` was never read.

This broke the build outright:

```
error MSB4018: Unable to find fallback package folder
'C:\Program Files (x86)\Microsoft Visual Studio\Shared\NuGetPackages'
```

A **Windows path inside a Linux container**. `Backend/*/obj/project.assets.json` is generated when you build on Windows and records your machine's NuGet folders. Because the Dockerfile passes `--no-restore`, the build trusted that stale file and went looking for a `C:\` path that cannot exist in Alpine Linux.

The same mistake hit the web build from the other side — `node_modules` uploaded as build context and would have overwritten the Linux packages `npm ci` had just installed. Angular's esbuild ships platform-specific native binaries.

The fix. One `.dockerignore` per build context. Measured effect: API context went from carrying `obj/` to **21 KB**; web context from **60 MB and climbing** to **867 KB**.

How to spot it yourself. Watch the `transferring context` line during a build. A .NET or Angular context should be a few MB. Past 50 MB, your `.dockerignore` is in the wrong place.

The nginx proxy

```
# No trailing slash on proxy_pass, so the /api prefix survives:
# /api/products -> http://api:8080/api/products
location /api/ {
    proxy_pass      http://api:8080;
    proxy_set_header Host      $host;
    proxy_set_header X-Forwarded-For  $proxy_add_x_forwarded_for;
}

# SPA fallback - a hard refresh on /catalog must return index.html,
# not a 404, so the Angular router can take over.
location / {
    try_files $uri $uri/ /index.html;
}
```

Two details that break things silently.

- **The trailing slash.** `proxy_pass http://api:8080;` keeps the `/api` prefix. Writing `http://api:8080/;` strips it and every call 404s.
- **The SPA fallback.** Without `try_files`, the app works when you click through it but 404s the moment anyone refreshes a deep link.

How the containers find each other

```
nginx -> http://api:8080      "api" = the service name
api    -> Host=db;Port=5432   "db"  = the service name
```

Never use `localhost` between containers. Inside a container, `localhost` means *that same container*.

Part II — PostgreSQL migration

Why

Neither Railway, Render, nor Vercel offers managed SQL Server. Running it as a container costs ~2 GB RAM permanently and the Developer edition is not licensed for production. PostgreSQL is first-class on all of them. The database was the only thing blocking deployment.

The audit came first

Before changing anything, seven parallel auditors examined the data layer. They returned **125 findings — 59 blockers and 81 silent-failure risks**. Three findings alone justified the exercise:

Trap	Why it would have shipped broken
Npgsql version	Every sibling EF package is <code>10.0.10</code> , so copying that number is the obvious move. But Npgsql's version does not track EF Core's patch number — the whole 10.x stable line is 10.0.0–10.0.3. <code>10.0.10</code> fails with <code>NU1102</code> .
bit lies	Hand-editing the migrations makes <code>nvarchar</code> and <code>datetime2</code> fail loudly — good. But <code>bit</code> is <i>valid PostgreSQL</i> meaning a 1-bit <i>bit string</i> , not a boolean. The table creates fine, then all 62 boolean columns mis-bind at runtime.
Identity annotations	<code>.Annotation("SqlServer:Identity", "1, 1")</code> is an <i>untyped string</i> . It survives package removal with no compile error, and Npgsql silently ignores unknown <code>SqlServer:</code> annotations. You get a schema that looks correct where every PK is a plain <code>integer</code> with no sequence — first insert into all 19 tables fails, in production, weeks later.

All three argue for the same thing: delete the migrations folder and regenerate. Never hand-patch a provider migration.

Decisions taken up front

#	Decision	Rationale
D1	Pin <code>Npgsql.EntityFrameworkCore.PostgreSQL</code> 10.0.3	Not 10.0.10, not a floating range. 10.0.3 constrains EF Core to [10.0.4, 11.0.0), which the existing graph satisfies.
D2	Delete all 11 migration files, scaffold one fresh <code>InitialCreate</code>	Zero <code>migrationBuilder.Sql()</code> , zero <code>HasData</code> , zero provider-specific config in the 14 entity Configurations — regeneration loses nothing.
D3	Start PostgreSQL empty	No production data exists. Eliminates the sequence-desync and timezone-shift traps entirely.
D4	Normalize-on-write, not <code>citext</code> or ICU collations	Also fixes the non-sargable <code>lower()</code> scans; needs no extension and no raw-SQL index migration.
D5	Database and role both lowercase <code>shopease</code>	Mixed case works through EF but folds the moment anyone types unquoted SQL in <code>psql</code> .
D6	Fix <code>DateTime</code> call sites and add a model-wide UTC converter	Deliberately <i>not</i> <code>EnableLegacyTimestampBehavior</code> , which reverts to naive timestamps across all 17 columns.

What changed, phase by phase

0 Baseline and teardown

```
docker compose down -v          # destroys the SQL Server volume
```

The old volume holds `.mdf` / `.ldf` files meaningless to Postgres, and Compose will happily reattach it.

1 PostgreSQL service and both connection strings

```
db:
  image: postgres:17-alpine
  environment:
    POSTGRES_USER: "shopease"
    POSTGRES_PASSWORD: "ShopEase#2026dev"
    POSTGRES_DB: "shopease"
  ports: ["5432:5432"]
  volumes: [pgdata:/var/lib/postgresql/data]
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U shopease -d shopease"]
```

The single most dangerous line in the compose file is the volume path. `postgres:17-alpine` starts perfectly happily with a volume mounted at the old `/var/opt/mssql`, initialises its data directory inside the container's writable layer, and **reports healthy**. Everything works. Every byte is gone on the next `docker compose down`.

The healthcheck must be `pg_isready`, not a port probe. During first-boot `initdb`, Postgres runs a temporary local-only server — a raw check on 5432 reports healthy while connections are still refused, and `api` gates on this. Both `-U` and `-d` are required for the same reason. The old `/dev/tcp` check could never pass on Alpine anyway (busybox `ash`, no `bash`).

Connection string keywords that fail in misleading ways

SQL Server	What happens under Npgsql
<code>Server=db,1433</code>	Parses successfully and wrongly . <code>Server</code> is a valid Npgsql alias for <code>Host</code> , and a comma-separated value is read as a multi-host failover list — so it tries to resolve a host literally named <code>1433</code> .
<code>MultipleActiveResultSets</code>	No equivalent. Parser throws <code>Keyword not supported</code> on first connection.
<code>TrustServerCertificate</code>	A lookalike. The official postgres image ships with TLS off entirely, so this is a false all-clear on a plaintext link.

Final value:

```
Host=db;Port=5432;Database=shopease;Username=shopease;Password=ShopEase#2026dev
```

`appsettings.Development.json` ships **inside the image** — neither `.dockerignore` excludes it. It is the live fallback the instant the compose env var is absent or misspelled, so it was migrated too. Left on LocalDB, the container would silently try to reach a Windows LocalDB instance.

2 Provider swap

```
- <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.10" />
+ <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.3" />
```

```
- options.UseSqlServer(cs));
+ options.UseNpgsql(cs, npgsql => npgsql.EnableRetryOnFailure());
```

This was the **only** `UseSqlServer` call site in the solution. This phase ends in a *deliberate* build failure — ~119 CS0103 errors inside the old migrations. That is the toolchain doing its job.

3 Regenerate the schema

```
dotnet ef migrations add InitialCreate -p ShopEase.Infrastructure -s ShopEase.Api -o Data/Migrations
```

11 files became 3. `-o Data/Migrations` matters — without it EF writes to `Migrations/` at the project root, breaking the existing layout and namespace.

No .NET SDK installed? Neither did this machine. Run EF inside the SDK container you already have:

```
docker run --rm -v "<project>\Backend:/src" -w /src mcr.microsoft.com/dotnet/sdk:10.0 \
  bash -c "dotnet tool install -g dotnet-ef --version 10.0.10 && \
    export PATH=\$PATH:/root/.dotnet/tools && \
    dotnet ef migrations add InitialCreate -p ShopEase.Infrastructure -s ShopEase.Api -o Data/Migrations"
```

Mount a copy **without** `obj/` and `bin/`, or the Windows-path problem from Part I returns.

The verification gate — actual results

Check	Expected	Got
nvarchar	0	✓ 0
datetime2	0	✓ 0
"bit" S-2	0	✓ 0
SqlServer: S-1	0	✓ 0
timestamp with time zone	present	✓ 51
boolean	present	✓ 30
numeric(18,2)	present	✓ 36
MaxIdentifierLength	63	✓ 63
Entities with identity	19	✓ 19
No identity on Roles.Id, CmsConfigs.Id, AppliedCoupons.UserId S-8	critical	✓ all three
Unique indexes	5	✓ 5

Why S-8 matters. Those three columns are `ValueGeneratedNever` . If the scaffold gave `AppliedCoupons.UserId` an identity, applying a coupon would write a *sequence-generated* `UserId` instead of the real one — wrong row, no error, ever.

4 DateTime.Kind

Npgsql maps `DateTime` to `timestampz` and **throws** on `Kind=Unspecified` or `Local`. EF + SQL Server was Kind-blind. Three seeder call sites produced `Unspecified`, all during Development boot.

```
- CreatedAt = DateTime.Parse(s.Created),  
+ CreatedAt = SeedUtc(s.Created),  
  
private static DateTime SeedUtc(string iso) =>  
    DateTime.SpecifyKind(DateTime.Parse(iso), CultureInfo.InvariantCulture), DateTimeKind.Utc);
```

Plus one systemic net covering all 17 `DateTime` properties across 13 entities — new file `Data/UtcDateTimeConverter.cs`, wired in via:

```
configurationBuilder.Properties<DateTime>().HaveConversion<UtcDateTimeConverter>();
```

The converter is what saves your existing backups. EF + SQL Server returned `datetime2` as `Kind=Unspecified`, so every backup JSON written before the migration serialises timestamps **without a Z**. Read back, they are `Unspecified` again and `SaveChangesAsync` throws. That hits 5 of the 6 restorable entities and is invisible until someone clicks Restore, possibly months later.

5 Case sensitivity — the phase people skip

SQL Server's default collation (`SQL_Latin1_General_CP1_CI_AS`) is **case-insensitive**. PostgreSQL is **case-sensitive**. There is no `HasCollation` anywhere in the solution, so every string column inherits the default. Four uniqueness guarantees silently weakened. Nothing errors, nothing fails to compile, the test suite stays green.

Pattern applied throughout: **normalize on write, normalize the input, compare with plain equality**. That restores the old semantics *and* makes the comparison index-seekable.

ID	Surface	What a user would have experienced
S-3	Email — <code>AuthService</code> , <code>UserRepository</code>	Priya registers as <code>Priya@Example.com</code> in March, again lowercase in August — the DB no longer stops her. Two rows. <code>FirstOrDefaultAsync</code> has no <code>ORDER BY</code> , so login resolves to <i>either</i> : her password intermittently fails, and when it works she may land in the wrong cart.
S-4	SKU — <code>ProductService</code> , <code>ProductRepository</code>	Two catalog entries for one physical product, split stock, double-counted inventory.
S-5	<code>Coupon.Code</code> (a string PK)	<code>save10</code> and <code>SAVE10</code> become two rows. Same code, different discount on different visits.
S-6	<code>CustomField.Entity</code>	Admin adds a second "Warranty" field; the duplicate probe misses, both rows get the identical Key, and one silently overwrites the other.
S-7	<code>BackupSnapshot.JobName</code>	A case-only rename orphans every earlier snapshot; retention silently stops working and the table grows without bound.

Your login query already used `ToLower()` on both sides, so authentication itself survived the migration. It was the duplicate rows that would have bitten — not the lookup.

6 Ordering determinism

SQL Server returned unordered queries in de-facto clustered-index order. PostgreSQL returns heap order, which changes after any `UPDATE` or `VACUUM`. Added `.ThenByDescending(x => x.Id)` to every timestamp sort and an explicit `OrderBy` where there was none — 19 call sites.

The one that mattered most: `BackupSnapshotRepository.TrimAsync` feeds `RemoveRange(snapshots.Skip(keep))`. An ordering flip on equal timestamps **deletes the wrong backup**.

Two sorts were collation-dependent and moved in-memory with `StringComparer.OrdinalIgnoreCase`: the backup-job list (under `LC_COLLATE=C` Postgres sorts `Zebra` before `apple`) and the brand facet list (SQL Server's CI collation collapsed `Sony` / `SONY` into one entry).

7 Seed and reset robustness

- `PaymentMethodId = (i % 3) + 1` hardcoded lookup ids. Those rows are inserted *without* explicit ids, and `Order.PaymentMethodId` is **not** an FK — so a mismatch corrupts silently. Now resolved by name.
- `ResetAllAsync`'s 15 sequential deletes had no transaction. On PostgreSQL each autocommits, so a failure partway left the database half-wiped. Now one transaction.

A subtlety worth knowing. `EnableRetryOnFailure` (added in phase 2) makes EF *reject* user-initiated transactions — "the configured execution strategy does not support user-initiated transactions". The transaction had to be handed to the execution strategy as a whole unit, because on retry it must replay all of it or none:

```
var strategy = _db.Database.CreateExecutionStrategy();
await strategy.ExecuteAsync(async () => { /* transaction here */ });
```

Verification — real output

Verified on this machine, 24 August 2026. All three containers up, shopease-db healthy, migration applied, 16 users / 20 products / 15 orders seeded.

Schema

Column	Type	Default
Id	integer	generated by default as identity
Email	text	
IsActive	boolean	
CreatedAt	timestamp with time zone	
Indexes:		
"IX_Users_Email" UNIQUE, btree ("Email")		

Behaviour

Test	Action	Result
S-3	Register Priya@Example.com	✓ 200, stored as priya@example.com
S-3	Register priya@example.com again	✓ 409 rejected
S-3	Log in as PRIYA@EXAMPLE.COM	✓ 200 + JWT
S-4	Create SKU el-001 where EL-001 exists	✓ 409 rejected, count stays 1
S-5	Apply coupon save10 five times	✓ resolves to SAVE10 every time
S-6	Custom field on Product then product	✓ second gets warranty_2
D4	explain analyze on email lookup	✓ Index Scan, was Seq Scan
S-10	down then up	✓ 16 users survived

The test suite proves nothing here. All 58 tests pass — but they use the InMemory provider, which has no collation, no DateTime.Kind validation and no SQL translation. They were green before the migration, they are green after, and they would stay green on a completely broken build. The evidence is the table above, not dotnet test.

Logging in

Email	Password	Role
admin@shopease.com	Admin@123	Admin — full dashboard

15 customers follow the pattern <firstname>@email.com / <Firstname>@123 — rahul, priya, arjun, sneha, vikram, anjali, karthik, divya, rohan, meera, aditya, pooja, sanjay, nisha, manoj.

Two accounts are deliberately inactive — karthik@email.com and pooja@email.com. They exist so you can demonstrate that a deactivated user is refused at login. If those two fail to sign in, that is the feature working.

Commands you will actually use

<code>docker compose up -d --build</code>	Build and start everything
<code>docker compose ps</code>	What is running right now
<code>docker compose logs -f api</code>	Follow one service's output
<code>docker compose down</code>	Stop — data survives
<code>docker compose down -v</code>	Stop and wipe the database
<code>docker exec -it shopease-db psql -U shopease -d shopease</code>	SQL prompt

In `psql`, quote your identifiers. EF creates quoted PascalCase tables, so `select * from users;` fails with relation "users" does not exist. You need `select * from "Users";`

Troubleshooting

Build fails with a `C:\` path inside the container

Host `obj/` reached the image. Add `bin / obj` to a `.dockerignore` inside the build context directory, not the repo root.

Disk fills mid-build

failed to solve: ... no space left on device

Free space, then `docker system prune -a` and `docker builder prune -a`. Restart Docker Desktop if the engine has gone unhealthy.

API keeps restarting

- **Keyword not supported** — a SQL Server keyword left in the connection string.
- **Cannot write DateTime with Kind=Unspecified** — a `DateTime.Parse` without `SpecifyKind`.
- **relation does not exist** — migration did not apply; check `docker compose logs api`.

Data vanished after `down`

Almost certainly the volume mount path. It must be `/var/lib/postgresql/data`. Any other path starts cleanly, reports healthy, and loses everything.

Deploying

	Vercel	Render	Railway
Runs .NET / Docker	✗	✓	✓
Managed Postgres	marketplace	✓ built in	✓
Free tier	✓ generous	✓ with caveats	trial credit

Recommended: Vercel for the frontend + Render for API and Postgres. Vercel's rewrites reproduce the nginx proxy exactly, so `apiUrl: '/api'` needs no change and CORS still never appears:

```
{ "rewrites": [{ "source": "/api/:path*",  
  "destination": "https://your-api.onrender.com/api/:path*" }] }
```

Two caveats on Render's free tier. Free web services sleep after 15 minutes idle with a ~50 second cold start. Free Postgres is **deleted** after 90 days — not suspended, deleted.

Rolling back

Full pre-migration copy:

```
C:\Users\1404\Downloads\shopease-BACKUP-2026-08-24-prePostgres
```

366 files. Excludes `node_modules`, `bin`, `obj`, which regenerate from lockfiles. The two original source files also have `.bak` siblings in place.